Project Report: A "Snake" Game in C with ‘raylib’ library

[Page 1]

1.0 Introduction and Project Goals

This report explains the design and logic for our "Snake" game project. The game was written entirely in C, using the raylib library for graphics and window management.[3]

When designing this project, we had three main goals to ensure the code was good for our computer science practices:

1. **Separate Logic from Drawing:** The game's rules (like how the snake moves) should be separate from the code that draws the snake on the screen.[2] This makes the code much easier to manage.
2. **No Global Variables:** A common problem in C projects is using global variables, which can make the program buggy and hard to debug.[4] Our goal was to avoid this by passing the game's state in a structured way.[3]
3. **Make it Efficient:** The 'Snake' game can get very slow if the snake's body is handled poorly. A major goal was to find a data structure that would keep the game running fast, even with a very long snake.[5]

This report will walk through how we used the raylib library, how we structured the game, and the logic behind the snake's movement, collisions, and growth.

[Page 2]

2.0 Technology: Why raylib?

2.1 Why raylib?

We needed a library to handle the basics: creating a window, drawing shapes, and getting keyboard input. We chose raylib for a few key reasons:

- **Simple and Easy:** raylib is famous for being simple and easy to use.[7] It's designed to get you programming without a lot of setup.
- **No Dependencies:** It's a single, self-contained library.[8] This made it very easy to compile and run our project without worrying about other complex libraries.

[Page 3]

3.0 Key raylib Functions Used

To build the game, we only needed a small number of raylib functions. This table shows the main functions we used and what they do.[3]

Table 1: Key Raylib API Functions Utilized in Project

Function	Purpose in Project
InitWindow	Creates the game window with our chosen size and title.[10]

SetTargetFPS	Asks raylib to run our game loop at 60 frames per second (FPS).[11]
WindowShouldClose	Checks if the user clicked the 'X' button or pressed the ESC key.[10]
BeginDrawing	Tells raylib we are about to start drawing a new frame.[10]
EndDrawing	Tells raylib we are finished drawing, and it shows the new frame on the screen.[10]
ClearBackground	Wipes the screen to a set color (like RAYWHITE) at the start of every frame.[10]
IsKeyPressed	Checks if a key was *just pressed* (fires only once). We use this for player turns.[10]
GetRandomValue	Gives us a random number, which we use to decide where to put the food.[11]
DrawRectangleV	The main drawing function. We use this to draw the snake's body parts and the food.[10]

[Page 4]

4.0 Core Game Structure

4.1 The Main Game Loop

The heart of our program is the main function, which contains the main game loop. This is a

standard pattern for all games.[12]

1. **Init:** First, we call InitWindow() and SetTargetFPS(60) to set up the game.[3]
2. **Loop:** We start a while (!WindowShouldClose()) loop. This loop will run 60 times every second until the player closes the game.[3]
3. **Update and Draw:** Inside the loop, we do two things, in order:
    - **Update:** We call our functions that handle game logic (like checking for input and moving the snake).
    - **Draw:** We call our functions that draw everything on the screen (between BeginDrawing() and EndDrawing()).[13]
4. **Close:** When the loop ends, we call CloseWindow() to clean up all resources.[3]

4.2 The State Machine (Handling "Screens")

To manage the different parts of the game (like playing, paused, or game over), we used a simple "Finite State Machine." This is a key CS concept and is very easy to build in C.

We first define an enum for our game's states 14:
typedef enum { STATE_PLAY, STATE_PAUSE, STATE_GAMEOVER } GameState;
Then, in our Update function, we use a switch statement to control the game's logic [15]:

C

```
// This is pseudocode for our update logic
void UpdateGame(Game *game) {
    switch (game->currentState) {
        case STATE_PLAY:
            // If we are playing, run the game logic
            UpdateGameplay(game);
            // Check if the 'P' key is pressed to pause
            if (IsKeyPressed(KEY_P)) game->currentState = STATE_PAUSE;
            break;

        case STATE_PAUSE:
            // If paused, do nothing! Just wait for 'P' again.
            if (IsKeyPressed(KEY_P)) game->currentState = STATE_PLAY;
            break;
```

```
        case STATE_GAMEOVER:
            // If game is over, wait for 'Enter' to restart
            if (IsKeyPressed(KEY_ENTER)) InitGame(game);
            break;
    }
}
```

This pattern is very powerful. To "pause" the game, we just stop running the UpdateGameplay function.[16] The drawing function still runs, so we can draw a "PAUSED" message on top of the frozen game.[3]

[Page 5]

5.0 The Timing Model: Separating Logic and Frames

A common problem in game programming is tying game speed to the frame rate.[17] If we move the snake "one square per frame," our game will run twice as fast on a 120 FPS monitor as on a 60 FPS monitor. This is unplayable.[18]

We also can't use "delta-time" (the time since the last frame), because 'Snake' is a grid-based game.[3] The snake must be *exactly* on (5, 5) or (5, 6), not on (5, 5.3).[3]

Our Solution: The "Tick" System

Our solution was to separate the "render rate" from the "logic rate".[3]

- **Render Rate (60 FPS):** The main loop runs 60 times per second, calling Update and Draw each time. This makes the game look smooth.[3]
- **Logic Rate (e.g., 6 TPS):** We only want the snake to move a few times per second.

We achieved this using a simple frame counter and the modulo operator (%).[19]

C

```
// Pseudocode for our main loop
int frameCounter = 0;
while (!WindowShouldClose()) {

    // --- UPDATE (This part runs 60 times per second) ---
    frameCounter++;
    PollInput(game); // Check for input every frame

    // --- LOGIC (This part ONLY runs every 10 frames) ---
    // (60 FPS / 10 = 6 moves per second)
    if (frameCounter % 10 == 0) {
        UpdateSnakeMovement(game);
        CheckCollisions(game);
    }

    // --- DRAW (This part runs 60 times per second) ---
    BeginDrawing();
    DrawGame(game);
    EndDrawing();
}
```

This is the best of both worlds. The game feels responsive and smooth, but the snake's speed is perfectly controlled and will be the same on *any* computer.[3] To change the difficulty, we just change 10 to a smaller number.

[Page 6]

6.0 Data Structures: The Core of the Program

6.1 Avoiding Global Variables

A key requirement for this project was to write clean, maintainable C code. In many simple C programs, people use global variables for things like snake_x, snake_y, and score.[4] This is dangerous because any function can change them, leading to bugs.[21]

Our solution was to use one main struct that holds *all* game data.[1]

C

```
// Simplified pseudocode for our structs
typedef struct {
    Vector2 body;
    int headIndex;
    int tailIndex;
    //...
} Snake;

typedef struct {
    Vector2 position;
    //...
} Food;

// The main struct holds everything!
typedef struct {
    Snake snake;
    Food food;
    int score;
    GameState currentState;
    //...
} Game;
```

In main(), we create one instance of this Game struct. Then, we pass a *pointer* to it (Game *game) to any function that needs it (like UpdateGame(Game *game) or DrawGame(Game *game)).[3] This makes our program's data flow very clear and prevents bugs.

6.2 The Snake Body Problem

The most important data structure is the one that holds the snake's body parts. A common-sense approach is to use an array:
Vector2 snakeBody;
But this has a big problem. When the snake moves, you have to shift every single element in the array [23]:

- snakeBody[1] = snakeBody
- snakeBody[2] = snakeBody[1]
- ...and so on, 254 more times!

This is an $O(n)$ operation. As the snake gets longer, the game will get slower and slower.[3]

[Page 7]

6.3 Our Solution: The Circular Buffer

We implemented a much more efficient data structure called a **Circular Buffer** (or circular queue).[24]

Instead of shifting the data, we just move our *pointers* to the data. We use a static array but also keep track of two integers: headIndex and tailIndex.[23]

- headIndex: The array index that holds the snake's head.
- tailIndex: The array index that holds the snake's tail.

The $O(1)$ Movement Algorithm:
When the snake moves, we don't shift any data. We just do this 3:

1. Calculate the newHeadPosition (e.g., one square to the right).
2. Increment the headIndex (and wrap it around with % MAX_SNAKE_LENGTH).
3. Store the newHeadPosition at body[headIndex].
4. Increment the tailIndex (and wrap it around).

That's it. The tail is "moved" simply by incrementing tailIndex, which leaves the old tail segment behind to be overwritten later. This takes the *exact same* amount of time whether the snake is 5 segments long or 500.

Table 2: Snake Body Data Structure Comparison

Data Structure	Move Algorithm	Speed (Time Complexity)	Our Rationale
Simple Array	Shift all n elements	**Slow ($O(n)$)**	Rejected: Too slow. Game lags as snake grows.[3]
Linked List	Add a new head node, free() the tail node	Fast ($O(1)$)	Rejected: malloc() and free() on every move is bad practice and can cause issues.[3]
Circular Buffer	Increment headIndex and tailIndex	**Fast ($O(1)$)**	**Implemented:** The perfect solution. It's $O(1)$ fast like a linked list but has no malloc/free overhead.[24]

[Page 8]

7.0 Core Game Mechanics

7.1 Input Handling

- **IsKeyPressed vs. IsKeyDown:** This was an important choice. IsKeyDown() is true for *every single frame* a key is held down.[26] If we used it, our snake would try to move 60 times a second. We use IsKeyPressed(), which fires *only once* when the key is first pressed.[10] This is perfect for our turn-based "tick" system.
- **180-Degree Turns:** We store the snake's current direction. When the player presses a key, we check if the new direction is the exact opposite of the current one. If it is (e.g., snake is moving RIGHT, player presses LEFT), we ignore the input to prevent the snake from instantly colliding with itself.[27]

7.2 Collision Detection

Because our game is on a grid, collision detection is simple. On every logic tick, we check the snake's new head position *before* we draw it:

1. Wall Collision: We check if head.x < 0, head.x >= GRID_WIDTH, head.y < 0, or head.y >= GRID_HEIGHT. If any of
these are true, we set the game state to GAME_OVER.30
2. **Self-Collision:** We loop through every active segment of the snake's body (from tailIndex to headIndex in our circular buffer). If the head's (x, y) coordinates are equal to any of the body's (x, y) coordinates, we set the game state to GAME_OVER.[27]

[Page 9]

7.3 Food Spawning

The food must appear on a random, empty tile. It cannot spawn *on* the snake. Our algorithm is a "random-until-safe" check [32]:

1. Create a do-while loop.
2. Inside the loop, get a random food.x and food.y using raylib's GetRandomValue().[33]
3. Set a isSafe flag to true.
4. Loop through the snake's body. If the food's position matches *any* snake body segment's

position, set isSafe to false and break the loop.[32]
5. The do-while loop's condition is while (!isSafe). If the spot was unsafe, the loop runs again, picking a new random spot.

7.4 Growth Mechanic

The growth logic is an elegant and simple part of our circular buffer design. When the snake's head position equals the food's position, we do two things:

1. Set a flag, like snake.isGrowing = true.
2. Call the SpawnFood() function to get a new food location.

Then, on the *very next* movement tick, the UpdateSnakeMovement function sees the isGrowing flag. When it does, it **skips incrementing the tailIndex** for that one move.[28]

The headIndex moves forward, but the tailIndex stays put. This
makes the snake's logical length permanently one segment longer. This is an $O(1)$ operation.

8.0 Rendering: From Grid to Pixels

Our game logic is completely separate from our drawing. The logic only knows about grid coordinates like (5, 5) or (10, 12).[35] The DrawGame function is responsible for translating these grid coordinates to pixel coordinates on the screen.

We do this with a simple SQUARE_SIZE constant (e.g., 40 pixels):

- pixel.x = grid.x * SQUARE_SIZE;
- pixel.y = grid.y * SQUARE_SIZE;

All drawing must happen inside a BeginDrawing() and EndDrawing() block.[10] The order matters for layering:

1. BeginDrawing();
2. ClearBackground(RAYWHITE); (Draw the background first)
3. Loop through the snake's body and draw each segment (e.g., DrawRectangleV(...))
4. Draw the food (DrawRectangleV(...))
5. Draw the UI / Score (DrawText(...)) (Draw the UI last so it's on top)
6. EndDrawing();

This keeps our logic clean (it doesn't know what a "pixel" is) and our rendering simple.

[Page 10]

9.0 Conclusion

This project was a success. We built a complete, working 'Snake' game from scratch in C.

By focusing on good CS principles, we created a program that is efficient and maintainable. The three most important design choices we made were:

1. **Using a Struct for Game State:** We successfully avoided global variables by putting all our data into a single Game struct and passing it by pointer.[1] This made our code clean and easy to follow.
2. **A Fixed Logic Tick:** We separated our game logic (6 moves per second) from our render rate (60 frames per second).[19] This ensures our game runs at the *exact same speed* on any computer.
3. **The Circular Buffer:** This was our best optimization. By using a circular buffer for the snake's body, we achieved an $O(1)$ (constant time) move operation.[24] This means our game *never* slows down, no matter how long the snake gets.

The combination of the C language and the simple raylib library allowed us to focus on important computer science concepts and build a solid, high-performance game.